Hotel Management & Guest Experience Platform

Functional Specification — what the website and admin panel do (v1.7 scope). Technology stack and visual design are intentionally out of scope; both are left to the development team.

---

1. Overview

This document describes a hotel website and admin platform covering three connected pieces: a public-facing site, a room booking flow with online payment, and a QR-code food ordering system for in-room and meeting-hall dining. It defines what the product must do — the business rules, user flows, data, and safeguards — without prescribing any particular technology or visual design.

---

2. Business Model

Room bookings are paid online at the end of the booking flow. On successful payment, the booking is confirmed automatically, a confirmation message is sent to the guest via a WhatsApp deep link, and a booking + payment notification email is sent to the hotel's configured email address with guest details, room, dates, meal add-ons, coupon applied, and amount paid.

QR-based food ordering stays cash-only — there is no online payment for food orders. Guests order via QR code, the order appears on the admin dashboard's live order list, and the hotel settles payment physically at the counter (cash/UPI/card, recorded manually by the admin). Order payment status is updated in the dashboard after settlement — it is never collected online.

- Order status, order-payment status, and booking status are three separate fields — never combined.
- Booking-payment status (online) and order-payment status (counter-settled) are tracked as distinct fields with distinct lifecycles — one enum is never reused for both.
- QR-based food ordering covers 40 rooms plus 1 meeting hall, with no guest login required.
- Room booking includes online payment, optional Meal Inclusion Plan (MIP) add-ons, and coupon code support.

The WhatsApp deep-link redirect sends a pre-filled order/booking message to the receptionist's WhatsApp. This is guest-triggered (the guest taps Send), not a server-pushed notification — so the admin panel's live order/booking list is the real source of truth reception should be watching, not WhatsApp. The hotel notification email is the reliable, server-pushed channel for bookings and should be treated as the authoritative record alongside the dashboard.

Roles

- **Admin** (owner/reception, single shared login): full access to everything — rooms, room types, menu, QR generation, bookings, orders, billing, invoicing, coupons, meal-plan pricing, payment settings, analytics, and audit log. There is a single admin role with no separate staff portal, no staff accounts, and no permission tiers.
- **Guest**: no account required. Browses the site, submits a room booking (pays online), scans a QR code to order food (pays at the counter), and tracks only their own booking/order status.

---

3. What the Platform Needs to Track (Core Entities)

The schema should be built around these entities and their relationships: **User (Admin), Hotel, RoomType, Room, Amenity, Booking, BookingGuest, MenuCategory, MenuItem, Order, OrderItem, Bill, Payment, QRCode, HotelSetting, AuditLog, Attraction, Coupon, MIPPlan, BookingMIPSelection.**

Payment (split by context)

A single payment record type is used for both bookings and food orders, distinguished by a **context** field (BOOKING or ORDER) — the two contexts never share fields or logic:

- **Booking payments** carry online-payment fields: gateway order ID, gateway payment ID, signature/verification reference, amount, currency, status, method, created-at, and verified-at timestamps.
- **Order payments** keep manual/counter fields: method (CASH/UPI/CARD/OTHER), amount, the admin who recorded it, and when it was recorded. Order payments never carry online-payment fields.

Coupon

Fields: code (unique, case-insensitive, e.g. RAAMA5), discount type (PERCENTAGE/FLAT), discount value, active flag, valid-from/valid-until dates, max uses (optional), times used, minimum booking amount (optional), and the admin who created it. Fully admin-managed via CRUD in the dashboard.

MIPPlan (Meal Inclusion Plan)

Admin-configured fixed pricing per meal type (BREAKFAST/LUNCH/DINNER), with a price and an active flag, editable in Settings. Prices are fixed by the admin — guests select which meals to add, they never enter a custom amount.

BookingMIPSelection

Joins a Booking to an MIPPlan: which meals were selected, the quantity (per guest or per night, per the meal-plan rules), and a snapshot of the price at the time of booking — so later price changes in Settings don't retroactively alter historical bookings.

---

4. Status Lifecycles

Entity	Lifecycle
Booking	PENDING → CONFIRMED → CHECKED_IN → CHECKED_OUT, plus CANCELLED / NO_SHOW
Order	PENDING → CONFIRMED → PREPARING → READY → DELIVERED, plus CANCELLED
Booking Payment (online, distinct enum)	PENDING → PAID, plus FAILED / REFUNDED. A booking must not move to CONFIRMED until its pa
Order Payment (counter-settled)	UNPAID / PARTIALLY_PAID / PAID
Room	AVAILABLE / RESERVED / OCCUPIED / CLEANING / MAINTENANCE / OUT_OF_SERVICE — no c

5. Non-Negotiable Business Rules

- Room identity from a QR scan is always validated server-side — never trust the URL parameter blindly, and a guest must never be able to view another room's orders, bills, or data by editing the URL.
- Booking availability is checked server-side with real transactional locking — no race-condition double bookings.
- QR order creation is never blocked or delayed by the WhatsApp step — the order is saved first, the WhatsApp redirect happens second.
- Booking creation is never blocked by the WhatsApp step either, but it is gated on successful, server-verified payment — no payment, no confirmed booking. A PENDING-payment booking record may briefly hold the room during checkout, but must auto-expire/release if payment isn't completed within a short, defined window (e.g. 10–15 minutes), so it doesn't block real availability.
- Guests never register or log in to order food or to book a room.
- PDF invoices use a human-readable invoice number, never a raw database ID.
- All payment amounts are computed and re-verified server-side at the moment of payment-order creation and again at verification — never trust a client-submitted amount, room price, meal-plan total, or coupon-discounted total (see Section 8).

6. Public Site — Pages & Content

- **Home** — hero section, a booking search entry point, featured rooms, a dining/menu teaser, a gallery teaser, a nearby-attractions teaser, and a booking call-to-action.
- **Rooms** — a browsable listing of room types, optionally filterable by capacity or price.
- **Room Details** — gallery, description, amenities, and a booking form for that specific room type. This is where meal-plan and coupon selection live in full (not just a quick entry point on Home).
- **Dining/Menu** — Veg / Non-Veg / Drinks category tabs.
- **Gallery** — bar and restaurant photos with a lightbox view.
- **Nearby Attractions** — a card grid (name, category, distance, photo, maps link), fully admin-managed.
- **About, Location, Contact** — standard informational pages.

7. Booking Flow (Room Reservation)

The booking form collects name, phone, email, check-in/check-out dates, number of guests, room type, and special requests, then proceeds through these steps:

- **1. Server-side availability check** for the selected room type and dates.
- **2. Meal Inclusion Plan (MIP) selection** — the guest optionally adds breakfast/lunch/dinner (priced per night or per guest, per admin-fixed pricing). The running total updates live but is always computed server-side at the point of order creation — never trusted from the client.

- **3. Optional coupon code entry** — validated server-side against the coupon rules (active, within date range, under max uses, meets minimum booking amount if set). Invalid or expired codes return a clear inline error — never silently ignored or silently applied.
- **4. Payment** — the server creates a payment order for the final, server-computed total (room + MIP – coupon discount); the guest completes payment through a hosted checkout so no card data ever touches the hotel's own servers.
- **5. On verified payment success** — the booking flips to CONFIRMED, the booking payment flips to PAID, the WhatsApp redirect fires with a pre-filled confirmation message, and a booking notification email is sent to the hotel's configured address with full booking, payment, meal-plan, and coupon detail.
- **6. On payment failure or cancellation** — the booking stays PENDING (or is released after the expiry window), the guest sees a clear retry option, and nothing is charged twice.

---

8. Guest QR Ordering Flow (Cash Only)

- A room-scoped menu page — no site header/footer chrome, optimized purely for fast mobile ordering.
- Cart → confirm → order created → WhatsApp redirect → order status tracker (Confirmed → Preparing → Ready → Delivered), scoped to that guest's own order only.
- No payment step of any kind here. The order arrives on the admin dashboard's live order list; the hotel settles cash/UPI/card at the counter; the admin marks the order's payment status in the dashboard afterward.

---

9. Admin Panel (Single Role, One Login)

- **Dashboard** — today's bookings/orders, pending items, occupied vs. available rooms, unpaid bills (both booking and order payments, clearly labeled by context), today's revenue (split by online booking payments vs. counter-settled order payments), and trend charts.
- **Bookings** — list + detail view, status transitions, payment status and transaction reference (read-only, sourced from the payment provider — never manually editable), meal-plan add-ons shown per booking, coupon applied shown per booking.
- **Orders** — live list with one-tap status transitions, manual payment recording (CASH/UPI/CARD/OTHER) and payment-status update, optional audio alert toggle for new orders.
- **Rooms & Room Types** — CRUD, room status management, QR generation/download per room.
- **Menu** — category and item CRUD.
- **Attractions** — CRUD for the Nearby Attractions content.
- **Coupons** — CRUD: code, discount type/value, active toggle, valid date range, max uses, minimum booking amount, and a read-only usage counter.
- **MIP Pricing** — set/edit the fixed price per meal type (breakfast/lunch/dinner), with an active toggle.
- **Billing** — bill breakdown, PDF invoice generation (including meal-plan line items and any coupon discount line), payment recording for orders (CASH/UPI/CARD/OTHER, supports partial payments). Booking payments here are read-only (the payment provider is the source of truth), with a manual "mark refunded" action tied to an actual refund, logged to the audit log.

- **Reports** — trends beyond the dashboard snapshot, including an online-payments view vs. a counter-cash view.
- **Settings** — hotel info, receptionist WhatsApp number, hotel notification email address, tax/service charge rate, payment gateway credentials (stored securely, server-side only — see Section 8), meal-plan pricing shortcut, coupon shortcut.
- **Audit log** — read-only history of key actions, including payment verification events, coupon creation/edits, and meal-plan price changes.

---

10. Payment Security Requirements (Non-Negotiable)

This section governs the room-booking payment flow only. QR food ordering never touches this section — it has no online payment surface at all.

Server-side trust boundary

- The client (browser) is never trusted to supply a final price. On the booking's final step, the server independently recomputes: room rate × nights, + meal-plan selections (using current pricing, snapshotted onto the booking at creation time), – coupon discount (re-validated server-side against live coupon rules), and creates the payment order for that server-computed amount only.
- Payment gateway API keys live server-side only — never shipped to the client bundle. Only a public/publishable key is passed to the client checkout widget; the secret never leaves the server.
- After checkout completes, the server must verify the payment before confirming the booking, using both (1) standard signature verification against the key secret, and (2) a webhook (with its own webhook secret) as the authoritative async confirmation — so a booking is never confirmed purely on a client-side callback that could be spoofed or interrupted mid-flow.
- A booking only transitions to CONFIRMED / Booking Payment PAID after webhook and signature verification both check out. If the client callback fires but the webhook hasn't landed yet, the guest sees a "confirming payment" state rather than a false success.

Abuse and integrity controls

- **Idempotency** — creating a payment order for the same booking attempt must be idempotent (keyed to a server-generated booking-attempt ID) so a guest double-tapping "Pay" or refreshing mid-flow cannot generate duplicate charges or duplicate bookings for the same room/dates.
- **Rate limiting** — booking-creation and payment-order-creation endpoints are rate-limited per IP/session to blunt scripted abuse and coupon brute-forcing.
- **Coupon brute-force protection** — failed coupon validation attempts are rate-limited/backed off per session; coupon codes are checked server-side only (never expose the full active coupon list to the client).
- **No amount-tampering surface** — the amount is never a hidden form field, query param, or anything else editable client-side that the server then trusts; it is recomputed fresh server-side at order-creation time, every time.
- **Held-room expiry** — a PENDING-payment booking places a short, time-boxed hold on the room; expired holds are released by a scheduled cleanup process so abandoned checkouts can't squat on inventory.

- **Refunds** are only ever initiated by the Admin from the dashboard, calling the provider's refund API server-side and writing an audit log entry — never a client-initiated refund path.
- **Webhook endpoint hardening** — the webhook route verifies the provider's signature header on every request and rejects anything that doesn't match before processing; it is never treated as trusted-by-default just because it's server-to-server.
- No card data ever touches the hotel's own infrastructure — checkout is hosted/embedded so card number/CVV never transits the hotel's servers, keeping PCI scope minimal by design.
- **Logging discipline** — log payment IDs, order IDs, statuses, and amounts for audit purposes; never log full webhook payloads containing sensitive tokens beyond what's needed, and never log secrets.
- HTTPS everywhere, enforced at the hosting layer, with no mixed content on payment pages.
- CSRF/session integrity on all state-changing booking and payment endpoints.

Note: This is a functional specification only. Technology stack, hosting, database choice, and visual/UI design are deliberately excluded — these decisions are left to the development team building the platform.